Introduction to Data Science

Web scraping and APIs

Simon Munzert
Hertie School | GRAD-C11/E1339

Table of contents

1. Scraping static webpages with R

2. Web scraping: good practice

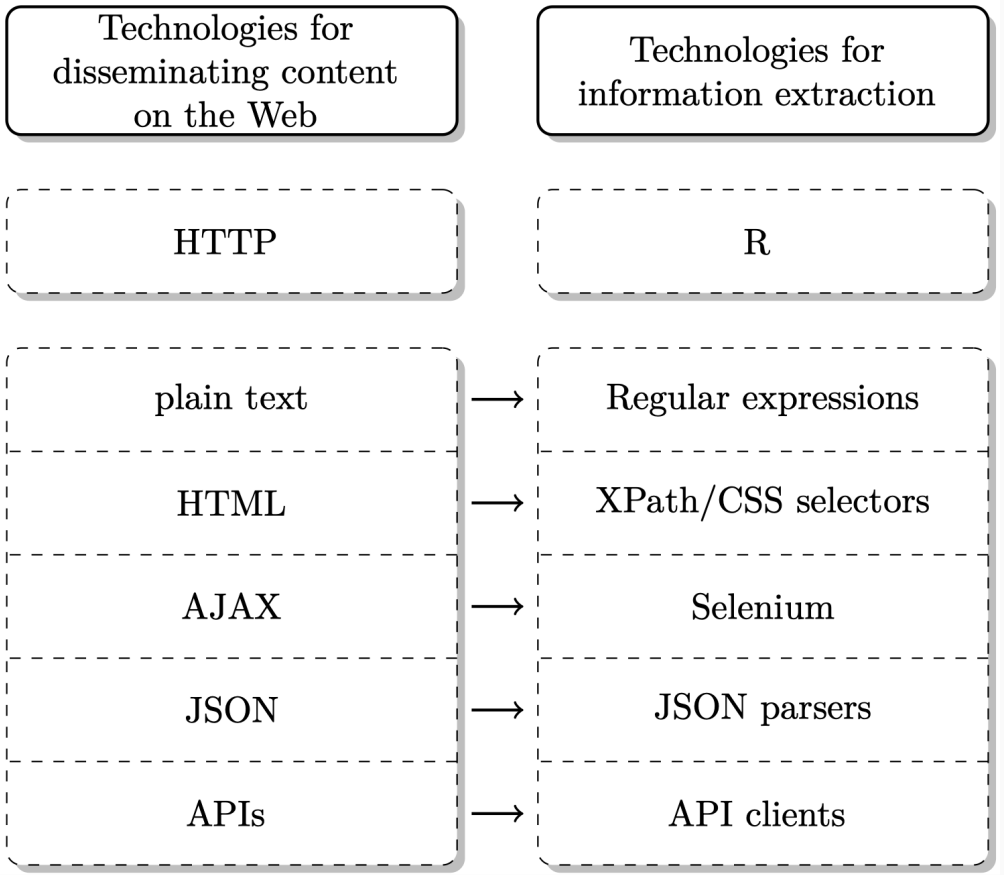
3. Web APIs: the basics

4. JSON

Scraping static webpages with R

Technologies of the world wide web

- To fully unlock the potential of web data for data science, we draw on certain web technologies.
- Importantly, often a basic understanding of these technologies is sufficient as the focus is on web data collection, not web development.
- Specifically, we have to understand
  - How our machine/browser/R communicates with web servers (→ **HTTP/S**)
  - How websites are built (→ **HTML**, **CSS**, basics of **JavaScript**)
  - How content in webpages can be effectively located (→ **XPath**, **CSS selectors**)
  - How dynamic web applications are executed and tapped (→ **AJAX**, **Selenium**)
  - How data by web services is distributed and processed (→ **APIs**, **JSON**, **XML**)



`Credit` ADCR

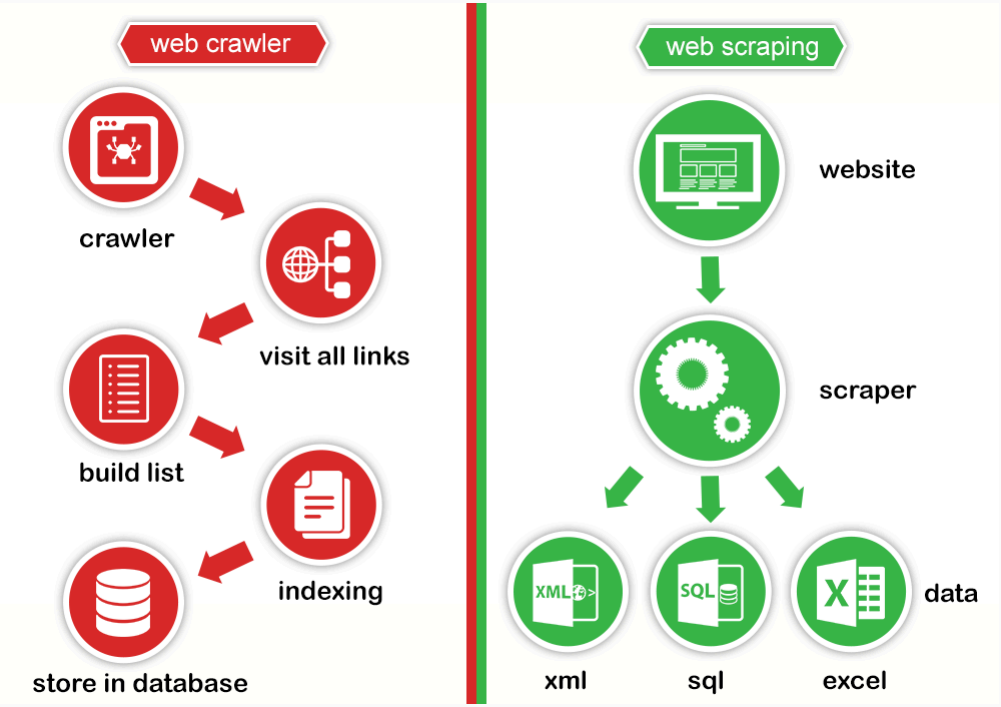
Web scraping

What is web scraping?

1. Pulling (unstructured) data from websites (HTMLs)
2. Bringing it into shape (into an analysis-ready format)

The philosophy of scraping with R

- No point-and-click procedure
- Script the entire process from start to finish
- Automate
  - The downloading of files
  - The scraping of information from web sites
  - Tapping APIs
  - Parsing of web content
  - Data tidying, text data processing
- Easily scale up scraping procedures
- Scheduling of scraping tasks



`Credit` prowebscraping.com

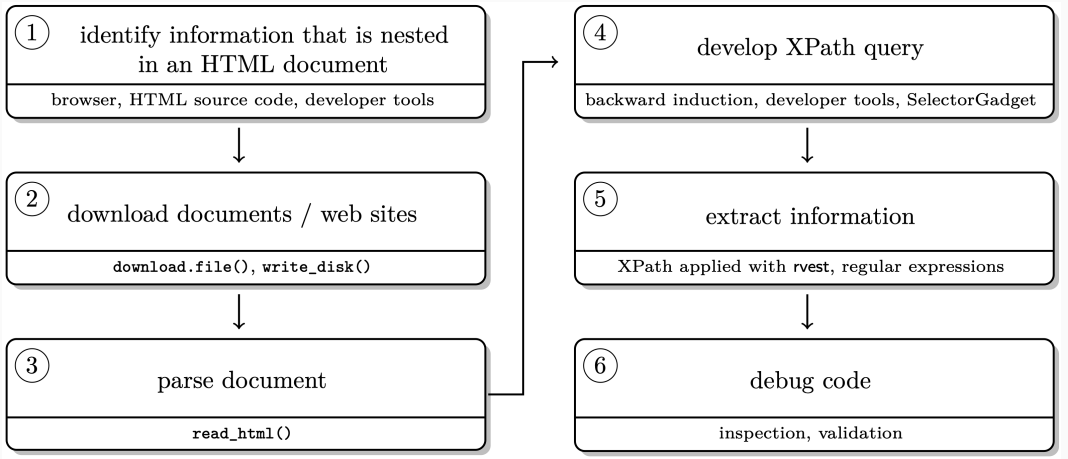
The scraping workflow

Key tools for scraping static webpages

1. You are able to inspect HTML pages in your browser using the web developer tools.
2. You are able to parse HTML into R with `rvest`.
3. You are able to speak XPath (or CSS selectors).
4. You are able to apply XPath expressions with `rvest`.
5. You are able to tidy web data with R/`dplyr`/`regex`.

The big picture

- Every scraping project is different, but the coding pipeline is fundamentally similar.
- The (technically) hardest steps are location (XPath, CSS selectors) and extraction (clean-up), sometimes the scaling (from one to multiple sources).



Web scraping with rvest

`rvest` is a suite of scraping tools. It is part of the tidyverse and has made scraping with R much more convenient.

There are three key `rvest` verbs that you need to learn.[1]

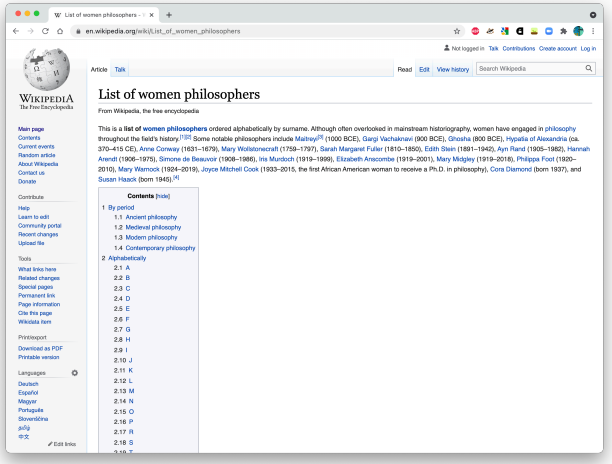
1. `read_html()`: Read (parsing) an HTML resource.
2. `html_elements()`: Find elements that match a CSS selector or XPath expression.
3. `html_text2()`: Extract the text/value inside the node set.

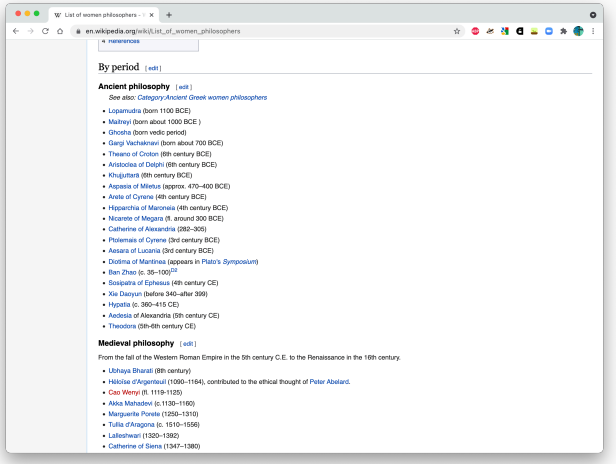


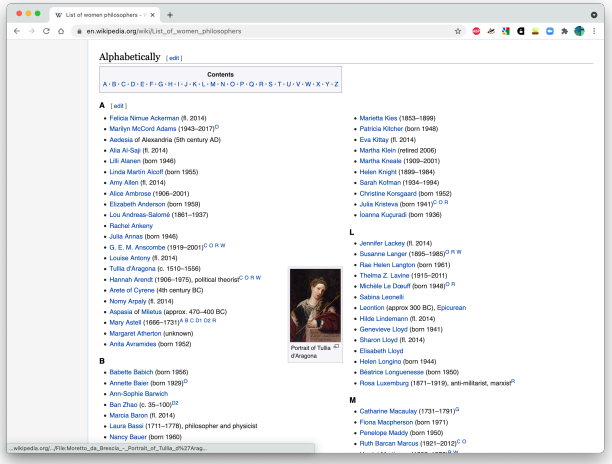
[1] There is more in `rvest` than what we can cover today. Have a glimpse at the overview at tidyverse.org and at this excellent (unofficial) cheat sheet.

Web scraping with rvest: example

- We are going to scrape information from a Wikipedia article on women philosophers available at https://en.wikipedia.org/wiki/List_of_women_philosophers.
- The article provides two types of lists - one by period and one sorted alphabetically. We want the alphabetical list.
- The information we are actually interested in - names - is stored in unordered list elements.








```
▶ <h2>…</h2>
▶ <div class="noprint">…</div>
▶ <h3>…</h3>
▶ <style data-mw-deduplicate="TemplateStyles:r998391716">
</style>
▼ <div class="div-col" style="column-width: 30em;">
  ▼ <ul>
    ▼ <li> == $0
        ::marker
        <a href="/wiki/Felicia_Nimue_Ackerman" title="Fel
        a Nimue Ackerman">Felicia Nimue Ackerman</a>
        " (fl. 2014)"
      </li>
    ▶ <li>…</li>
    ▶ <li>…</li>
    ▶ <li>…</li>
```

Scraping with rvest: example (cont.)

Step 1: Parse the page

```
R> url_p <- read_html("https://en.wikipedia.org/wiki/List_of_women_philosophers")
```

Step 2: Develop an XPath expression (or multiple) that select the information of interest and apply it

```
R> elements_set <- html_elements(url_p, xpath = "//h2[text()='Alphabetically']//following::li/a[1]")
```

The XPath expression reads:

- `//h2`: Look for `h2` elements anywhere in the document.
- `[text()='Alphabetically']`: Look for `h2` elements with the content `"Alphabetically"`.
- `//following::li`: In the DOM tree following that element (at any level), look for `li` elements.
- `/a[1]` within these elements look for the first `a` element you can find.

Scraping with rvest: example (cont.)

Step 3: Extract information and clean it up

```
R> phil_names <- elements_set |> html_text2()
R> phil_names[c(1:2, 101:102)]
```

```
## [1] "A"                    "B"                    "Elisabeth of Bohemia"
## [4] "Dorothy Emmet"
```

Step 4: Clean up (here: select the subset of links we care about)

```
R> first <- which(str_detect(phil_names, "Felicia Nimue Ackerman"))
R> last  <- which(str_detect(phil_names, "Alenka Zupan"))
R> philosopher_names_clean <- phil_names[first:last]
R> length(philosopher_names_clean)
```

```
## [1] 267
```

```
R> philosopher_names_clean[1:5]
```

```
## [1] "Felicia Nimue Ackerman" "Marilyn McCord Adams"   "Aedesia"
## [4] "Alia Al-Saji"           "Lilli Alanen"
```

Scraping HTML tables

Purchased Equipments (June, 2006)			
Item Num#	Item Picture	Item Description	Price
		Shipping Handling, Installation, etc	Expense
1.		IBM Clone Computer.	$ 400.00
		Shipping Handling, Installation, etc	$ 20.00
2.		1GB RAM Module for Computer.	$ 50.00
		Shipping Handling, Installation, etc	$ 14.00
Purchased Equipments (June, 2006)			

Built	Building	City	Country	Roof		Floors	Pinnacle		Current status
1870	Equitable Life Building	New York City	United States	043 m	142 ft	8			Destroyed by fire in 1912
1889	Auditorium Building	Chicago		082 m	269 ft	17	106 m	349 ft	Standing
1890	New York World Building	New York City		094 m	309 ft	20	106 m	349 ft	Demolished in 1955
1894	Philadelphia City Hall	Philadelphia		155.8 m	511 ft	9	167 m	548 ft	Standing
1908	Singer Building	New York City		187 m	612 ft	47			Demolished in 1968
1909	Met Life Tower			213 m	700 ft	50			Standing
1913	Woolworth Building			241 m	792 ft	57			Standing
1930	40 Wall Street					70	283 m	927 ft	Standing
1930	Chrysler Building			282.9 m	927 ft	77	319 m	1,046 ft	Standing
1931	Empire State Building			381 m	1,250 ft	102	443 m	1,454 ft	Standing
1972	World Trade Center (North Tower)			417 m	1,368 ft	110	527.3 m	1,730 ft	Destroyed in 2001 in the September 11 attacks
1974	Willis Tower (formerly Sears Tower)	Chicago		442 m	1,450 ft	108	527 m	1,729 ft	Standing
1996	Petronas Towers	Kuala Lumpur	Malaysia	379 m	1,242 ft	88	452 m	1,483 ft	Standing
2004	Taipei 101	Taipei	Taiwan	449 m	1,474 ft	101	509 m	1,671 ft	Standing
2010	Burj Khalifa	Dubai	United Arab Emirates	828 m	2,717 ft	163	829.8 m	2,722 ft	Standing

DATES	POLLSTER	GRADE	SAMPLE	WEIGHT	APPROVE	DISAPPROVE	ADJUSTED	
DEC. 28-30	Gallup	B-	1,500 A	1.03	40%	55%	41%	53%
DEC. 26-28	Rasmussen Reports/Pulse Opinion Research	C+	1,500 LV	0.85	45%	53%	40%	53%
DEC. 24-28	Ipsos	A-	1,519 A	2.01	37%	58%	37%	57%
DEC. 23-27	Gallup	B-	1,500 A	0.58	38%	56%	39%	54%
DEC. 24-26	YouGov	B	1,500 A	1.13	38%	52%	39%	55%

Scraping HTML tables

- HTML tables are everywhere.
- They are easy to spot in the wild - just look for `<table>` tags!
- Exactly because scraping tables is an easy and repetitive task, there is a dedicated `rvest` function for it: `html_table()`.

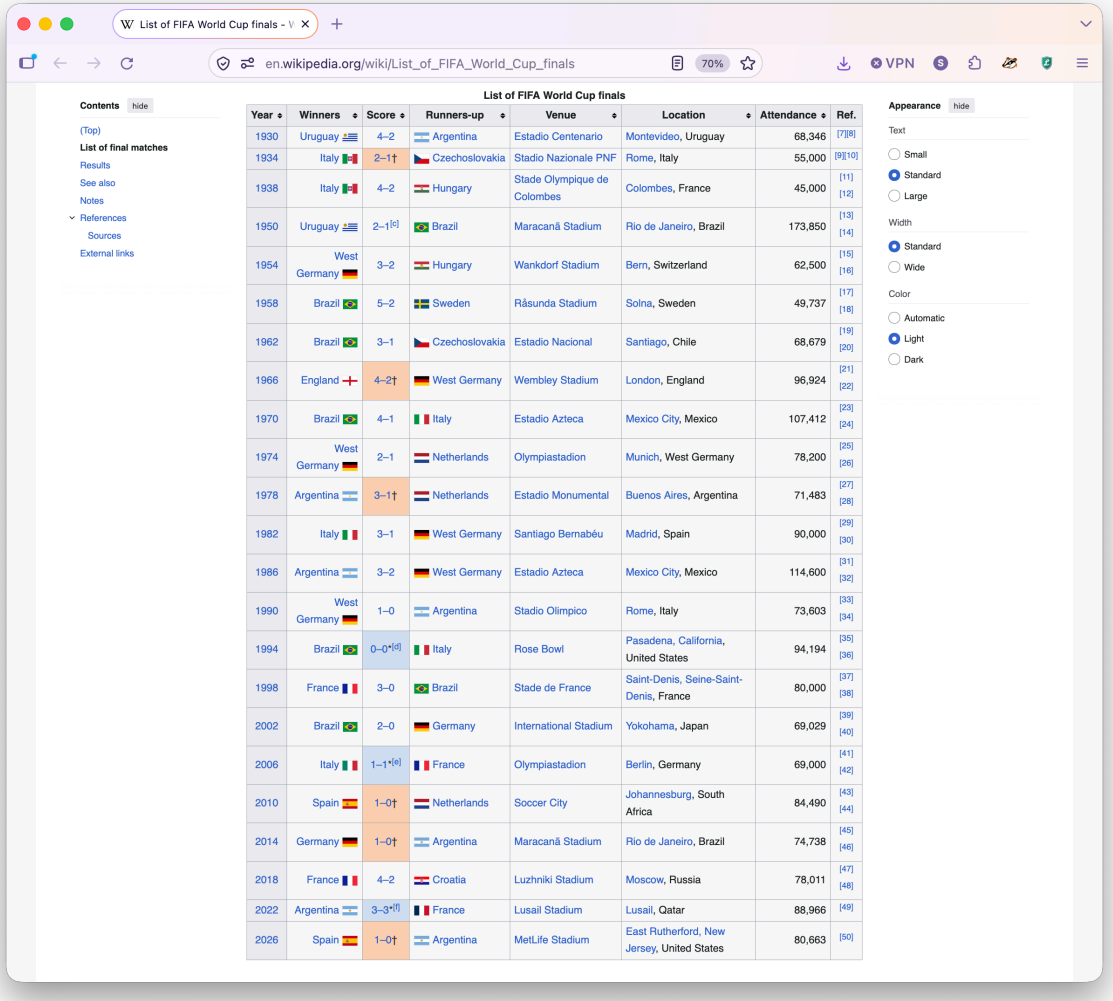
Function definition

```
R> html_table(x,
+     header = NA,
+     trim = TRUE,
+     dec = ".",
+     na.strings = "NA",
+     convert = TRUE
+ )
```

Argument	Description
`x`	Document (from `read_html()`) or node set (from `html_elements()`).
`header`	Use first row as header? If `NA`, will use first row if it consists of `<th>` tags.
`trim`	Remove leading and trailing whitespace within each cell?
`dec`	The character used as decimal place marker.
`na.strings`	Character vector of values that will be converted to `NA` if `convert` is `TRUE`.
`convert`	If `TRUE`, will run `type.convert()` to interpret texts as int, dbl, or `NA`.

Scraping HTML tables: example

- We are going to scrape a small table from the Wikipedia page List_of_FIFA_World_Cup_finals.
- (We're actually using a pinned revision from October 2026, accessible here. Wikipedia pages change, this revision won't.)
- The table is not entirely clean: The score column mixes numbers with symbols and footnote markers, and there are flag icons, links, and a references column.
- The HTML code looks straightforward though: a `<table>` with one `<tr>` per final.



Scraping HTML tables: example

- We are going to scrape a small table from the Wikipedia page List_of_FIFA_World_Cup_finals.
- (We're actually using a pinned revision from October 2026, accessible here. Wikipedia pages change, this revision won't.)
- The table is not entirely clean: The score column mixes numbers with symbols and footnote markers, and there are flag icons, links, and a references column.
- The HTML code looks straightforward though: a `<table>` with one `<tr>` per final.



Scraping HTML tables: example (cont.)

```
R> library(rvest)
R> url <- "https://en.wikipedia.org/wiki/List_of_FIFA_World_Cup_finals"
R> url_p <- read_html(url)
R> # Wikipedia's data tables carry the class "wikitable"; the first one here is a one-row legend
R> tables <- url_p |> html_elements("table.wikitable") |> html_table(header = TRUE)
R> fifa_finals <- tables[[2]]
R> head(fifa_finals)
```

```
## # A tibble: 6 × 8
##    Year Winners      Score  `Runners-up`   Venue       Location Attendance Ref.
##   <int> <chr>        <chr>  <chr>          <chr>       <chr>    <chr>      <chr>
## 1  1930 Uruguay      4–2    Argentina      Estadio Ce… Montevi… 68,346     [7][…
## 2  1934 Italy        2–1†   Czechoslovakia Stadio Naz… Rome, I… 55,000     [9][…
## 3  1938 Italy        4–2    Hungary        Stade Olym… Colombe… 45,000     [11]…
## 4  1950 Uruguay      2–1[c] Brazil         Maracanã S… Rio de … 173,850    [13]…
## 5  1954 West Germany 3–2    Hungary        Wankdorf S… Bern, S… 62,500     [15]…
## 6  1958 Brazil       5–2    Sweden         Råsunda St… Solna, … 49,737     [17]…
```

Which table is the right one you have to find out by inspecting the page or the output of `html_table()`. The cleanup (scores, attendance, footnote markers) is part of the live coding session; see `materials/fifa-world-cup-finals.R`.

Websites have braced against being scraped

What used to work

- For years, point-and-click helpers such as SelectorGadget (a bookmarklet/browser extension that generates XPath or CSS selectors from the elements you click) made scraping static pages a breeze.
- Many of the examples I used in this course a few years ago were built that way. **Most of them don't return anything anymore.**

What changed

- **Auto-generated class names.** Front-end frameworks (React, styled-components, Tailwind, …) create hashed classes like `erslblw0` at build time. They carry no meaning and change with every redeployment. Selectors built on them break within weeks.
- **Content Security Policies (CSP).** Sites now restrict which scripts may run on their pages. Bookmarklets and injected helper scripts are blocked, so tools like SelectorGadget often cannot even start.
- **JavaScript-rendered content.** The HTML the server sends is often an empty shell. The content is loaded and rendered client-side, so `read_html()` sees nothing.
- **Anti-bot infrastructure.** Rate limits, bot-detection challenges (e.g., Cloudflare), logins, and paywalls.

Why this is partly on purpose

- Hashed class names are a side effect of modern tooling. Content has become valuable (not least as AI training data) and site owners guard it.

Lessons learned

- **Writing XPath by hand is still a relevant skill.** Prefer selectors that rest on the semantics of the page (element names, structure, `id`s, readable classes such as `.teaser__headline`, `data-*` attributes) over generated class names. The browser's developer tools are your friend; CSP does not block them.
- **Expect your scrapers to break.** Pin the page version you worked with (e.g., Wikipedia's `oldid`), store raw HTML locally, and let your code fail loudly when a selector returns nothing.
- **Look for the data behind the page.** JavaScript-rendered content usually comes from an API the page calls in the background (watch the `Network` tab). That API is often easier to tap than the page itself (more on APIs in a minute).

Web scraping: good practice

Scraping: the rules of the game

1. You take all the responsibility for your web scraping work.
2. Think about the nature of the data. Does it entail sensitive information? Do not collect personal data without explicit permission.
3. Take all copyrights of a country's jurisdiction into account. If you publish data, do not commit copyright fraud.
4. If possible, stay identifiable. Stay polite. Stay friendly. Obey the scraping etiquette.
5. If in doubt, ask the author/creator/provider of data for permission—if your interest is entirely scientific, chances aren't bad that you get data.

Consult robots.txt

What's robots.txt?

- "Robots exclusion standard", informal protocol to prohibit web robots from crawling content
- Located in the root directory of a website (e.g., google.com/robots.txt)
- Documents which bot is allowed to crawl which resources (and which not)
- Not a technical barrier, but a sign that asks for compliance

How is it structured?

- Not an official W3C standard
- Rules listed bot by bot
- General rule listed under `User-agent: *` (most interesting entry for R-based crawlers)
- Directories folders listed separately

Example

```
User-agent: Googlebot
Disallow: /private/
```

Universal ban

```
User-agent: *
Disallow: /
```

Allow declaration

```
User-agent: *
Disallow: /pics/
Allow: /pics/public/
```

Crawl delay (in seconds)

```
User-agent: *
Crawl-delay: 2
```

Downloading HTML files

Stay modest when accessing lots of data

- Content on the web is publicly available.
- But accessing the data causes server traffic.
- Stay polite by querying resources as sparsely as possible.

Two easy-to-implement practices

1. Do not bombard the server with requests - and if you have to, do so at modest pace.
2. Store web data on your local drive first, then parse.

Looping over a list of URLs

```
R> for (i in seq_along(list_of_urls)) {
+   if (!file.exists(paste0(folder, file_names[i])))
+     download.file(list_of_urls[i],
+                   destfile = paste0(folder, file_n
+                   )
+     Sys.sleep(runif(1, 1, 2))
+     }
+ }
```

- `!file.exists()` checks whether a file does not exist in the specified location.
- `download.file()` downloads the file to a folder. The destination file (location + name) has to be specified.
- `Sys.sleep()` suspends the execution of R code for a given time interval (in seconds).

Staying identifiable

Don't be a phantom

- Downloading massive amounts of data may arouse attention from server administrators.
- Assuming that you've got nothing to hide, you should stay identifiable beyond your IP address.

Two easy-to-implement practices

1. Get in touch with website administrators / data owners.
2. Use HTTP header fields `From` and `User-Agent` to provide information about yourself (by passing these to `add_headers()` from the `httr` library).

Staying identifiable in practice

```
R> url <- "http://a-totally-random-website.com"
R> rvest_session <- session(url,
+  httr::add_headers(From = "my@email.com",
+                `User-Agent` =
+                  R.Version()$version.string))
R> headlines <- rvest_session |>
+               html_elements(xpath = "p//a") |>
+               html_text2()
```

- `rvest`'s `session()` creates a session object that responds to HTTP and HTML methods. (Under the hood it uses the `httr` package, hence `httr::add_headers()`.)
- Here, we provide our email address and the current R version as `User-Agent` information.
- This will pop up in the server logs: The webpage administrator has the chance to easily get in touch with you.

Scraping politely with the `polite` package

Etiquette as code

The `polite` package by Dmytro Perepolkin bundles the good practices from the previous slides into three verbs:

- `bow()`: Introduce yourself to the host. Declares your user agent, fetches and parses `robots.txt`, and checks whether you are allowed to scrape the path.
- `nod()`: Switch to another path on the same host (re-checks permission).
- `scrape()`: Fetch the page. Respects the crawl delay from `robots.txt` (default: 5 seconds), caches responses so that repeated calls don't hit the server again, and returns the parsed HTML for `rvest`.

Install

The package is currently not on CRAN. Install it from GitHub:

```
R> remotes::install_github("dmi3kno/polite")
```



Example

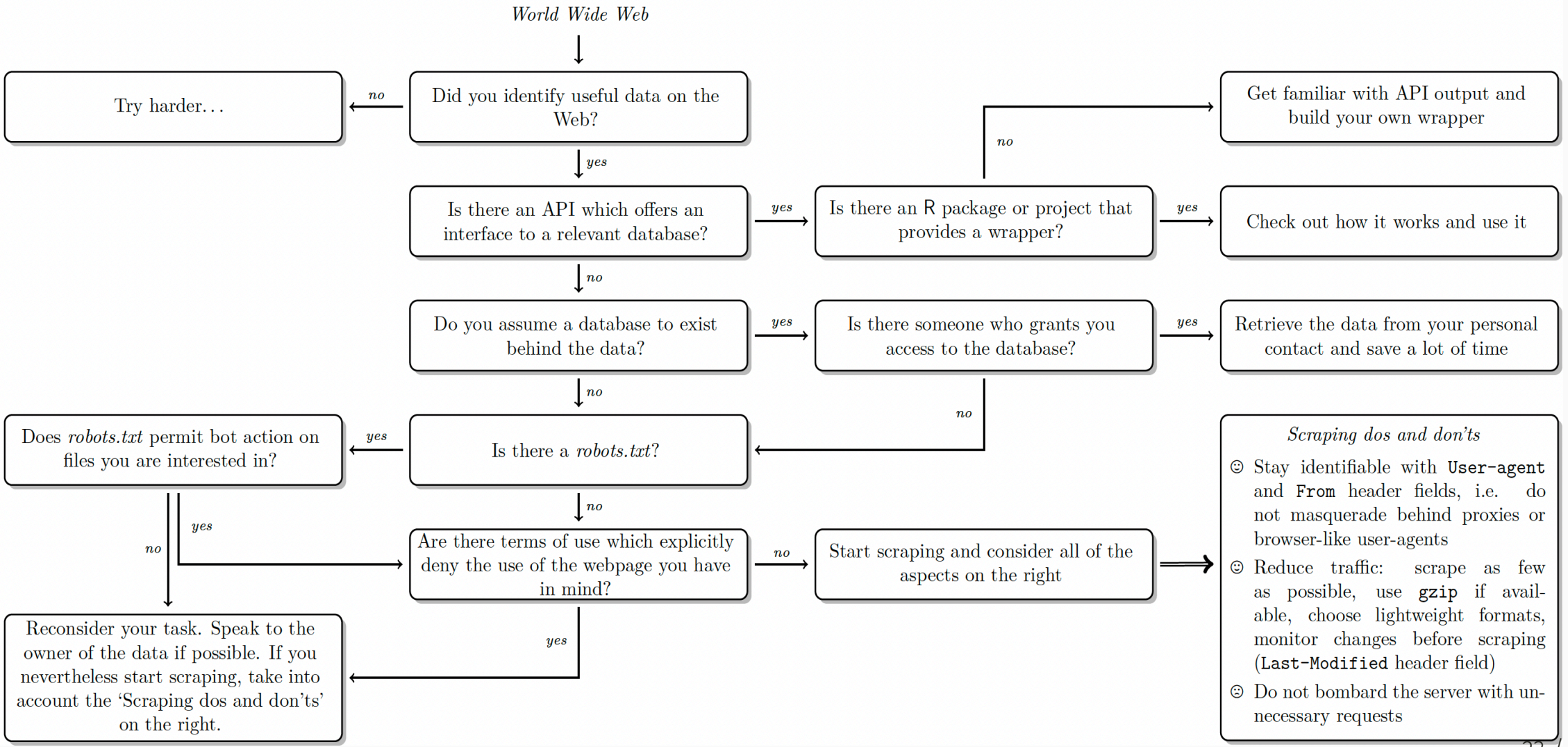
```
R> library(polite)
R> session <- bow(
+   "https://en.wikipedia.org/wiki/List_of_FIFA_World_Cup_finals",
+   user_agent = "IDS course (munzert@hertie-school.org)"
+ )
R> session
```

```
## <polite session> https://en.wikipedia.org/wiki/List_of_FIFA_World_Cup
##     User-agent: IDS course (munzert@hertie-school.org)
##     robots.txt: 464 rules are defined for 34 bots
##    Crawl delay: 5 sec
##   The path is scrapable for this user-agent
```

```
R> url_p <- scrape(session)
R> tables <- url_p |> html_elements("table.wikitable") |> html_tab
R> tables[[2]] |> head(3) |> select(1:4)
```

```
## # A tibble: 3 × 4
##    Year Winners Score `Runners-up`
##   <int> <chr>   <chr> <chr>
## 1  1930 Uruguay 4–2   Argentina
## 2  1934 Italy   2–1†  Czechoslovakia
## 3  1938 Italy   4–2   Hungary
```

Scraping etiquette (cont.)



Scraping in the age of AI

AI crawlers have changed the game

- Training large language models requires huge amounts of web text. AI companies run crawlers (`GPTBot`, `ClaudeBot`, `Google-Extended`, …) at a scale that dwarfs academic scraping.
- Many websites now explicitly block these bots in their `robots.txt` (see the community-maintained ai.robots.txt list), add anti-bot measures (Cloudflare challenges, rate limits), or move content behind logins.
- Side effect for us: Scraping has become harder, and site owners are more suspicious of *any* automated traffic. Being polite and identifiable matters more than ever.
- The legal debate (copyright, terms of use) is in flux. Research purposes are often privileged (e.g., § 60d UrhG in Germany), but check before you publish data.

LLMs as scraping assistants

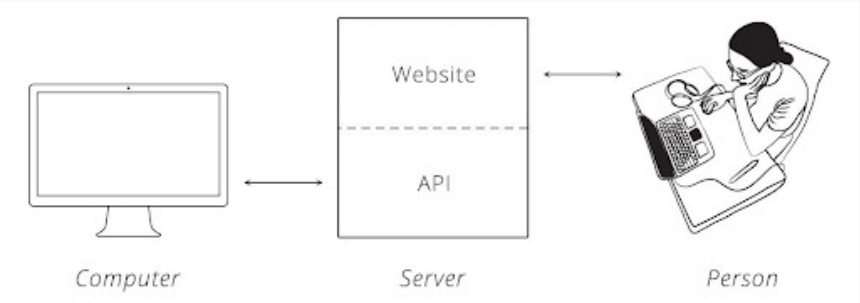
- LLMs are good at writing XPath/CSS selectors and `rvest` code when you paste them a chunk of HTML. Use them to speed up the location and extraction steps.
- They are also useful for the tidying step: turning messy, unstructured text into structured records (names, dates, amounts).
- Beware though:
  - Generated selectors are only as good as the HTML snippet you provided. Always test on the full page and on multiple pages.
  - LLM-based extraction is non-deterministic and can hallucinate. Validate against ground truth, and prefer a deterministic parser whenever the structure allows it.
  - Pasting scraped content into a third-party model may itself violate terms of use or data protection rules.

Web APIs: the basics

What are web APIs?

Definition

- A web **A**pplication **P**rogramming **I**nterface lets you/your program query a provider for specific data.
- Think of web APIs as "data search engines": You pose a request, the API answers with a bulk of data.
- Many popular web services provide APIs (Google, Wikipedia, OpenAlex, the Bundestag, ...).
- Often, APIs provide data in JSON, XML (can be any format though).



`Credit` Brian Cooksey

Key concepts

- **Server:** A computer that runs an API and can be talked to.
- **Client:** A program that exchanges data with a server through an API.
- **Protocol:** The rule set underlying how computers talk to each other (e.g. HTTP).
- **Method:** The "verbs" that clients use to talk with a server (in HTTP speak: `GET`, `POST`, and others).
- **Endpoint:** URLs that can be specified in a particular way to query the API database.
- **Request:** What the client asks of the server (see Methods above).
- **Response:** The server's response. This includes a *Status Code* (e.g. "200", "404"), a *Header* (meta-information about the reponse), and a *Body* (the actual content that we're interested in).

What do APIs do?

Key perks from a web scraping perspective

- APIs provide instance access to clean data.
- They free us from building manual scrapers.
- They make it easier for a computer to interact with data on server.
- API usage implies mutual agreement about data collection.



Restaurant analogy[1]

Restaurant	Web API
You, the client	Your program/computer
The restaurant	API provider
The waiter	API
The menu	API documentation
Your order	Specified API endpoint (request)
The kitchen	API database
The food	API response
The bill	API pricing

[1] Kitchen example inspired by Jason Johl

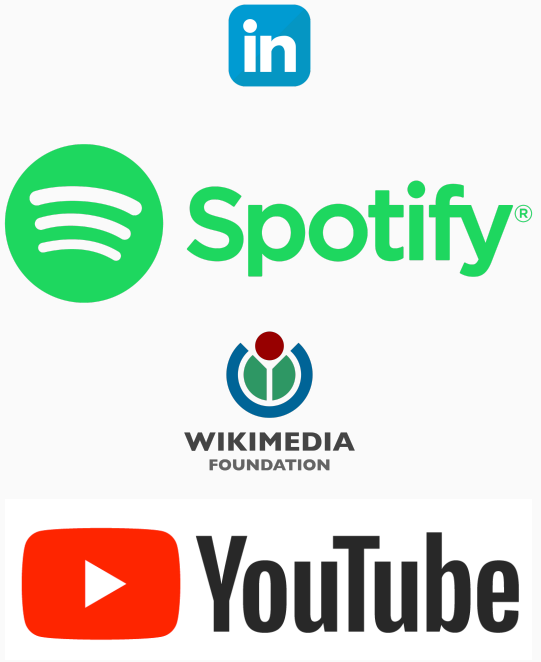
Why do organizations have APIs?

Scalability and regulation of access

- Imagine if everyone decided to retrieve data from a server in an unstructured way. The amount of energy that this would consume would be very high.
- Also, access to data would be largely unregulated.
- With APIs, organizations can provide a regulated and organized way for clients to retrieve a large amount of data, without overwhelming or crashing their server, or violating their Terms of Use.

More reasons

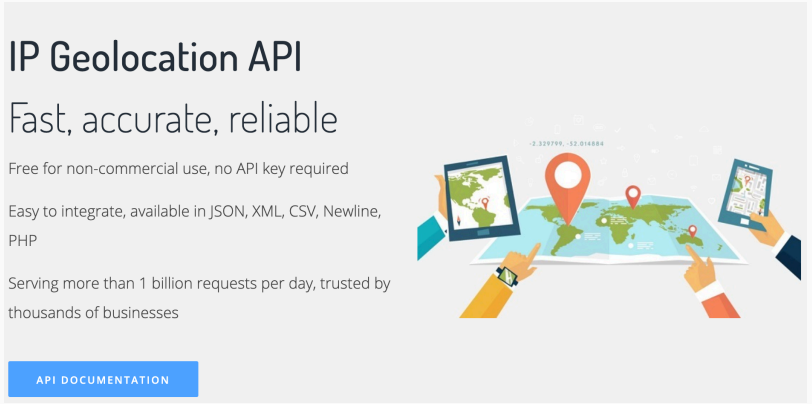
- **Monetization:** Data can be turned into a sellable product.
- **Innovation:** Clients have access to data and develop their own products and solutions.
- **Expansion:** APIs can help companies move to different markets or partner with other companies.



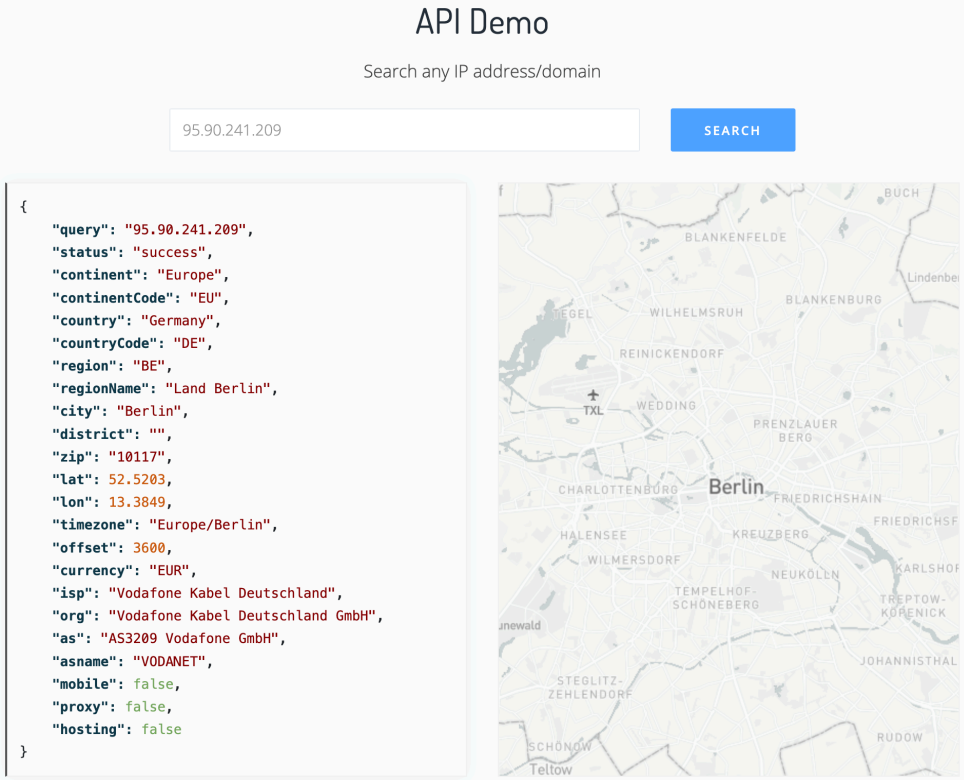
Example: Working with the IP API

The IP API

- The IP API at https://ip-api.com/ takes IP addresses and provides geolocation data (latitude, longitude, but also country state, city, etc.) in return.
- This is useful if you want to, e.g., map IP address data (although this is not perfectly accurate).
- The API is free to use and requires no registration. (There's also a pro service for commercial use though.)



An example JSON response



Example: Documentation

Overview

Overview

This documentation is intended for developers who want to write applications that can query IP-API.
We serve our data in multiple formats via a simple URL-based interface over HTTP, which enables you to use our data directly from a user's browser or from your server.

Geolocation API

Response formats

JSON

XML

CSV

Newline

PHP

Batch API

Query multiple IP addresses in one HTTP request.

Batch JSON

JSON endpoint

The API base path is

`http://ip-api.com/json/{query}`

{query} can be a single IPv4/IPv6 address or a domain name. If you don't supply a query the current IP address will be used.

Parameters

Query parameters (such as custom fields and JSONP callback) are appended as GET request parameters, for example:
`http://ip-api.com/json/?fields=61439`

fields	response fields *optional*
lang	response language *optional*
callback	wrap inside (JSONP) *optional*

There is no API key required.

Quick test

You can edit this query and experiment with the options

GET http://ip-api.com/json/24.48.0.1 SEND

Example: Calling the API with R

First, we specify the endpoint. We use the JSON endpoint and start with an empty query field.

```
R> endpoint <- "http://ip-api.com/json"
```

Next, we build a request with `httr2`'s `request()` and send it with `req_perform()`. (The default method is `GET`.)

```
R> library(httr2)
R> response <- request(endpoint) |> req_perform()
R> response
```

```
## <httr2_response>
## GET http://ip-api.com/json
## Status: 200 OK
## Content-Type: application/json
## Body: In memory (404 bytes)
```

Hooray, we successfully called the API and got JSON data in return! We extract the body with `resp_body_json()`.

```
R> response_parsed <- resp_body_json(response)
R> str(response_parsed)
```

```
## List of 14
##  $ status     : chr "success"
##  $ country    : chr "Germany"
##  $ countryCode: chr "DE"
##  $ region     : chr "BE"
##  $ regionName : chr "State of Berlin"
##  $ city       : chr "Berlin"
##  $ zip        : chr "10117"
##  $ lat        : num 52.5
##  $ lon        : num 13.4
##  $ timezone   : chr "Europe/Berlin"
##  $ isp        : chr "Verein zur Foerderung eines Deuts
##  $ org        : chr "Hertie School of Governance gGmbH
##  $ as         : chr "AS680 Verein zur Foerderung eines
##  $ query      : chr "195.37.184.82"
```

Example: Calling the API with R *cont.*

This looks like an R list, which is useful. `resp_body_json()` parsed the JSON response into an R list. We could have also kept the raw text with `resp_body_string()`.

For low-dimensional data like this, the `jsonlite` package can do the whole job (request + parse) in one go:

```
R> response_parsed <- jsonlite::fromJSON(endpoint)
R> response_parsed
```

```
## $status
## [1] "success"
##
## $country
## [1] "Germany"
##
## $countryCode
## [1] "DE"
##
```

Given that the data structure is low-dimensional, we can easily map it onto a data frame:

```
R> as.data.frame(response_parsed)
```

```
##    status country countryCode
## 1 success Germany          DE
##   region      regionName
## 1     BE State of Berlin
##     city   zip     lat
## 1 Berlin 10117 52.5125
##       lon      timezone
## 1 13.3893 Europe/Berlin
##
## 1 Verein zur Foerderung eines Deutschen Forschungsnetz
##                                             org
## 1 Hertie School of Governance gGmbH, Berlin
##
## 1 AS680 Verein zur Foerderung eines Deutschen Forschun
##           query
## 1 195.37.184.82
```

Example: Calling the API with R *cont.*

We can easily modify the call to retrieve more data:

```
R> endpoint <- "http://ip-api.com/json/91.198.174.1"
R> response_parsed <- jsonlite::fromJSON(endpoint)
R> response_parsed
```

```
## $status
## [1] "success"
##
## $country
## [1] "The Netherlands"
##
## $countryCode
## [1] "NL"
##
## $region
## [1] "NH"
##
## $regionName
## [1] "North Holland"
##
## $city
```

The API also allows for batch processing. This gives you the ability to query multiple IP addresses in one HTTP request, which is significantly faster than submitting individual queries. To that end, the batch of IP addresses or domains must be sent using a `POST` request. With `httr2`, adding a JSON body via `req_body_json()` switches the method to `POST` automatically:

```
R> endpoint <- "http://ip-api.com/batch"
R> response <- request(endpoint) |>
+   req_body_json(list("208.80.152.201", "91.198.174
+   req_perform()
R> resp_body_string(response) |>
+   jsonlite::fromJSON() |>
+   as.data.frame()
```

```
##    status         country countryCode region    regionN
## 1 success   United States          US     CA    Califo
## 2 success The Netherlands          NL     NH North Holl
##       lat        lon             timezone
## 1 37.7892 -122.40200 America/Los_Angeles Wikimedia Fou
```

Accessing APIs with R

Is there a pre-built API client for R?

1. **Yes.** Great. Use it if it provides the functionality you need (if not, see option 2).
2. **No.** Build your own API client.



API clients

- Provide an interface to APIs
- Hide the API back-end
- Let you stay in your programming environment
- Good news: In most cases, this will be the world you live in
- Please always cite package authors when you use their work. Run `citation("<package name>")` to see how.

Building a client from scratch

- Dive into the API documentation
- Write your own functions to specify endpoint calls and turn API output into R data structures
- Build infrastructure for API authentication if necessary

API clients: example

The `ipapi` package by Bob Rudis provides high-level access to the IP API. You just have to install it and can then work with the `geolocate()` function provided by the package to call the API.

```
R> # remotes::install_github("hrbrmstr/ipapi") # uncomment to install if necessary
R> library(ipapi)
R> ip_df <- geolocate(c("", "10.0.1.1", "www.spiegel.de"), .progress=TRUE)
R> ip_df
```

```
##     status country countryCode region      regionName              city    zip
##     <char>  <char>      <char> <char>          <char>            <char> <char>
## 1: success Germany          DE     BE State of Berlin            Berlin  10117
## 2:    fail    <NA>        <NA>   <NA>            <NA>              <NA>   <NA>
## 3: success Germany          DE     HE           Hesse Frankfurt am Main  60314
##        lat      lon      timezone
##      <num>    <num>        <char>
## 1: 52.5125 13.38930 Europe/Berlin
## 2:      NA       NA          <NA>
## 3: 50.1124  8.71858 Europe/Berlin
##                                                          isp
##                                                       <char>
## 1: Verein zur Foerderung eines Deutschen Forschungsnetzes e.V.
## 2:                                                       <NA>
```

Restricted API access

Why API access can be restricted

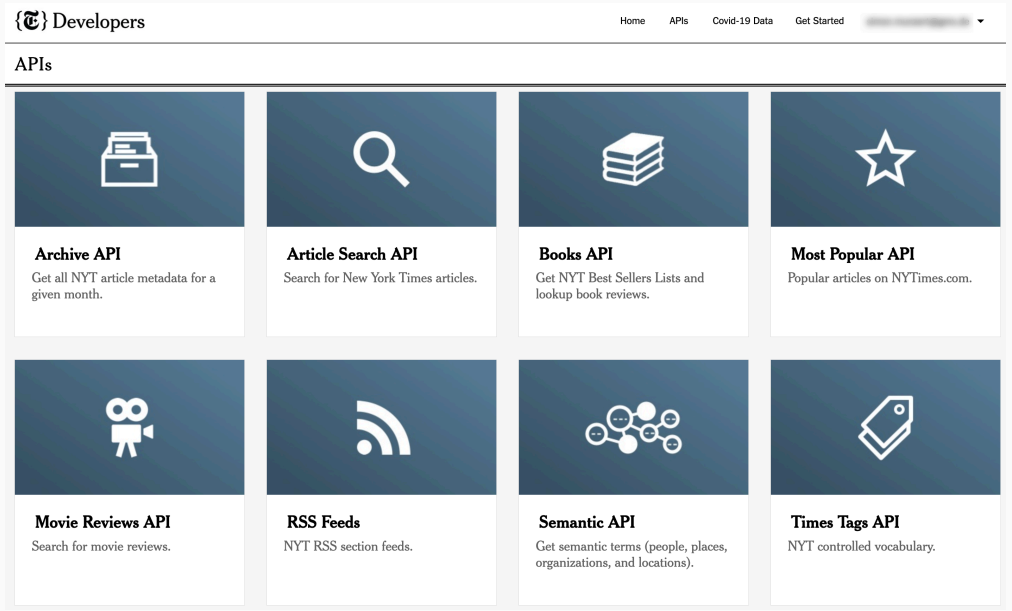
- The service provider wants to know who uses their API.
- Hosting APIs is costly. API usage limits can help control costs.
- The API hoster has a commercial interest: You pay for access (sometimes for advanced features or massive queries only).

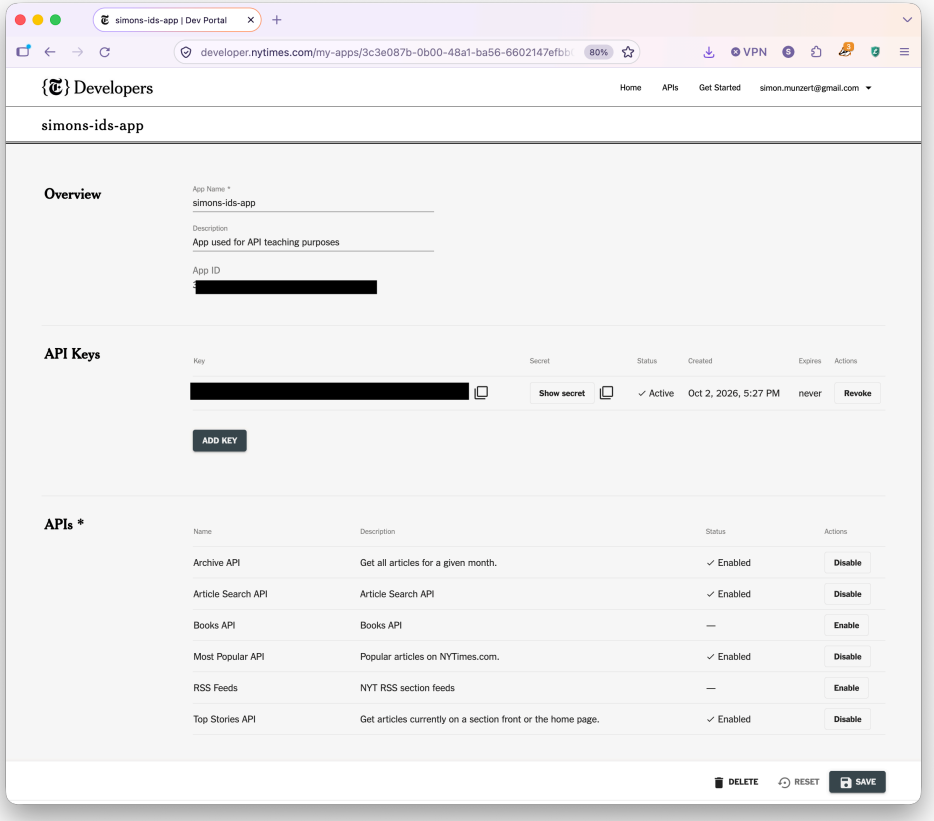
Access tokens

- Access tokens serve as keys to an API.
- They usually come in form of a randomly generated string, such as `dk5nSj485jJZP3847kjU`.
- Obtaining a token requires registration; sometimes payment. Sometimes disclosure of your intentions.
- Once you have the token, you pass it along with your regular API call using a key parameter.

Restricted API access: example

The *New York Times* provides several APIs for developers at https://developer.nytimes.com/. In order to use them, we have to register as a developer (for free) and register an app (here: `simons-ids-app`). The app gets assigned a key, which we can use to call one of the APIs. (Note: NYT deactivates keys that aren't used for a while, so don't be surprised if you have to regenerate it.)





Restricted API access: example *cont.*

First, we load the key that we stored separately as a string (here: `nytimes_apikey` stored in `rkeys.RDa`, outside the project folder).

```
R> load("/Users/s.munzert/rkeys.RDa")
```

Next we build the request. The key is passed as a URL query parameter, which we add with `req_url_query()`:

```
R> endpoint <- "https://api.nytimes.com/svc/mostpopular/v2/viewed/1.json"
R> req <- request(endpoint) |> req_url_query(`api-key` = nytimes_apikey)
```

Finally, we call the API and inspect the results:

```
R> nytimes_most_popular <- req |> req_perform() |> resp_body_json(simplifyVector = TRUE)
R> nytimes_most_popular$results$title[1:3]
```

```
## [1] "California Prepares to Ban Quartz Countertop Production"
## [2] "What, Exactly, Did Trump Say About Iran Taking Out Los Angeles?"
## [3] "How to Destroy a Reputation in Hollywood, as Told in 5 Women's Lawsuits"
```

Restricted API access: some advice

NEVER hard-code your personal API key (or any personal information for that matter) in R scripts that you plan to share.

Instead, use one of the following options:

1. Store your API keys in a separate file that you store somewhere else and only import for the purpose of using them (see previous example).
2. Store your API keys in environment variables.

For the second option, you can use `Sys.setenv()` and `Sys.getenv()` as in:

```
R> ## Set new environment variable called MY_API_KEY. Current session only. Don't store code in script.
R> Sys.setenv(MY_API_KEY="abcdefghijklmnopqrstuvwxyz0123456789")
R>
R> ## Assign the environment variable to an R object and pass it on where needed.
R> my_api_key = Sys.getenv("MY_API_KEY")
```

The downside of this approach is that this environment variable will not persist across sessions. To make it persistent (on your system), you should modify the `.Renviron` file (`usethis::edit_r_environ()` opens it for you). Check out these instructions to learn how that works.

Recap: tapping APIs with R

1. Figure out whether an API is available that serves your needs.

2. Figure out whether an up-to-date and fully functional R client for API is available. Make yourself familiar with the API Terms of Use and client functionality, then use it.

3. If no client is available, build your own.

   a. Dive into the documentation.

   b. Use the `httr2` package to construct and perform requests (`request()`, `req_url_query()`, `req_body_json()`, `req_perform()`).

   c. Use `resp_body_json()` or the `jsonlite` package to parse the JSON response (or other parsers such as `xml2`, depending on API output).

   d. Address API error handling, user agent, authentication, rate limiting, pagination issues. `httr2` has helpers for all of these (`req_retry()`, `req_user_agent()`, `req_auth_*()`, `req_throttle()`, `req_perform_iterative()`). See here for more info.

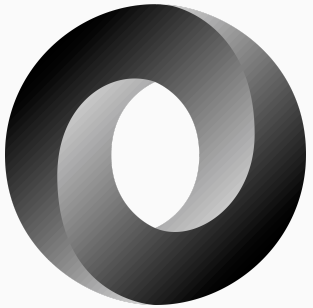
   e. Write useful high-level functions that wrap your API calls and parsing operations.

JSON

What's JSON?

Quick facts

- **J**ava**S**cript **O**bject **N**otation
- Popular data exchange format for web services / APIs
- "the fat-free alternative to XML"
- JSON ≠ Java, but a subset of JavaScript
- However, very flexible and not dependent upon any programming language
- Import of JSON data into R is relatively straightforward with the `jsonlite` package





Example

```
[
   {
      "name" : "van Pelt, Lucy",
      "sex" : "female",
      "age" : 32
   },
   {
      "name" : "Peppermint, Patty",
      "sex" : "female",
      "age" : null
   },
   {
      "name" : "Brown, Charlie",
      "sex" : "male",
      "age" : 27
   }
]
```

Basic syntax

Types of brackets

1. Curly brackets, `{` and `}`, embrace objects. Objects work similar to elements in XML/HTML and can contain other objects, key-value pairs or arrays.
2. Square brackets, `[` and `]`, embrace arrays. An array is an ordered sequence of objects or values.

Data structure

JSON can map complex data structures (objects nested within objects etc.), which can make it difficult to convert JSON into flattened R data structures (e.g., a data frame).

Also, there is no ultimate JSON-to-R converting function.

Luckily, JSON files returned by web services are usually not very complex. And `jsonlite` simplifies matters a lot.

Key-value pairs

Keys are put in quotation marks; values only if they contain string data.

```
"name" : "van Pelt, Lucy"
"age" : 32
```

Keys and values are separated by a colon.

```
"age" : 32
```

Key-value pairs are separated by commas.

```
{"name" : "van Pelt, Lucy", "age" : 32}
```

Values within arrays are separated by commas.

```
["van Pelt, Lucy", "Peppermint, Patty"]
```

JSON and R

Parsing JSON with R

- There are different packages available for JSON parsing with R.
- Choose `jsonlite` by Jeroen Ooms: It's well maintained and provides convincing mapping rules.

Key functions

There are two key functions in `jsonlite`:

- `fromJSON()`: converts input from JSON data into R objects following a set of conventions.
- `toJSON()`: converts input from R objects into JSON data

Get started with the package following this vignette.

Conversion rules of `jsonlite`

```
R> library(jsonlite)
R> x <- '[1, 2, true, false]'
R> fromJSON(x)
```

```
## [1] 1 2 1 0
```

```
R> x <- '["foo", true, false]'
R> fromJSON(x)
```

```
## [1] "foo"   "TRUE"  "FALSE"
```

```
R> x <- '[1, "foo", null, false]'
R> fromJSON(x)
```

```
## [1] "1"     "foo"   NA      "FALSE"
```

More on web scraping

Until now, the toy scraping examples were limited to single HTML pages. However, often we want to **scrape data from multiple pages**. You might think of newspaper articles, Wikipedia pages, shopping items and the like. In such scenarios, automating the scraping process becomes really powerful. Also, principles of polite scraping are more relevant then.

In other cases, you might be confronted with

- forms,
- authentication,
- dynamic (JavaScript-enriched) content, or want to
- automatically navigate through pages interactively.

There's only so much we can cover in one session. Check out more material online here and there to learn about solutions to some of these problems.

More on web APIs

Collecting data from the web using APIs provided by the data owner represents the **gold standard of web data retrieval**. It allows for pure data collection without "HTML layout waste", standardized data access, de facto agreement on data collection by the data owner, and robustness and scalability of data collection.

On the other hand, the rise of API architectures is not without issues. It requires knowledge of the architecture and creates dependencies on API suppliers. While APIs have the potential to make data access more democratic, they can also add to the siloing of information.

There are many resources that give an overview of existing public APIs, including this List of public APIs on GitHub. Another useful resource is APIs for social scientists - a collaborative review.

Finally, if you plan to write an R client for a web API, check out this guide.